Herald

Herald — MVP Specification (V1, superseded)

Field	Value
Revision	3
Created	2026-05-15
Last modified	2026-05-19
Status	superseded
Status summary	Superseded by `specification.V2.md` on 2026-05-19. Preserved for historical reference and as the source of the R-NN recommendation IDs that V2 references.
Issues	none
Issues summary	—
Fixed	R-01, R-04, R-09, R-10, R-15, R-19, R-20, R-21, R-22 (text-level fixes carried into V2)
Fixed summary	text-level recommendations applied; heavier R items folded into V2 sections (architecture, security, observability, supply chain)
Continuation	See `specification.V2.md` — substantive next round: populated TBDs, added flavors (`bherald/dherald/aherald/scherald/iherald/rherald/cherald`), new sections §"Event model & wire format" (CloudEvents), §"Architecture overview" (Watermill + River), §"Channel addressing" (Apprise-style URLs + tags), §"Multi-tenancy & Isolation" (Postgres RLS + Redis ACL), §"Observability & SLOs" (OpenTelemetry), §"Supply chain & release engineering" (SLSA L3), §"Extensibility" (subprocess+manifest deferred)

The bi-directional ingesting system events and reliably fanning them out to multiple notification channels so every alert reaches the right destination without confusion.

Table of contents

Upstreams

All existing project upstreams:

- **GitHub** (main repository): `git@github.com:vasic-digital/Herald.git`
- **GitLab**: `git@gitlab.com:vasic-digital/herald.git`
- **GitFlic**: `git@gitflic.ru:vasic-digital/herald.git`
- **GitVerse**: `git@gitverse.ru:vasic-digital/Herald.git`

What Herald can (must be able to) do?

Herald is the mechanism (system) which receives input from the source(s) and sends it to one or more destinations (implemented channels). Depending on the implementation (Flavor) of the Herald sources and destinations could be various. We can have single type or multiple type inputs and same applies for the outputs.

For example, input can be the result of execution of some pipeline. Herald is then sending this result (some report for example) to certain output (or outputs). It could be, for a sake of illustration, a messaging system which notifies subscribers.

The Possibilities are not limited!

The structure of the System MUST be hierarchical so in the top levels (closest to the root) we have abstractions and base reusable, main shared mechanisms, shared codebase, while inside the `flavors` directory we MUST have all flavors - the implementations.

> **Note:** We MUST NOT be obligated to follow this structure as many parent project's specific custom flavors may need to exist! They could be private and require safe location in the System or project. We MUST make sure that such flexibility is possible! This MUST BE fully supported!

How Herald should achieve its mission objectives?

Every herald flavor will be individual binary which users can add to the System path through the `.bashrc` or `.zshrc` (for example). Each Herald flavor will have shared set of commands and parameters while there will be as well commands and parameters specific to particular Herald flavors (implementations).

Herald applications are CLI binaries which are mainly designed for CI integration and various Pipelines. They can be easily incorporated for use with various AI CLI Agents (Claude Code, OpenCode and others...) as well or other similar use cases (triggering by `cron` jobs, and so on ...).

Some Herald application names can be: `pherald` for the `Project Herald`, `sherald` for the `System Herald` and so on.

Integration into the Constitution

Once the whole project is fully implemented, tested and verified with proof(s) and confirmation of complete anti-bluff validation and verification we MUST promote candidate universal rules into the root Constitution, `AGENTS.md` and `CLAUDE.md` **via a HelixConstitution PR** (audited per Universal §11.4 + §11.4.17 — universal-vs-project classification), never by editing the parent `constitution` Submodule (`git@github.com:HelixDevelopment/HelixConstitution.git`) from inside Herald (see R-09 and `CONSTITUTION_INHERITANCE.md` §"Promoting Herald rules into the constitution"). Each Flavor (see below) will present its Constitution extensions through the same promotion process.

> **Note:** Herald's implementation MUST BE in direct connection with the `constitution` Submodule!

See also: `../../guides/HERALD_CONSTITUTION.md` (Herald's already-adopted project articles, extending Helix Universal) and `../../guides/CONSTITUTION_INHERITANCE.md` (discovery contract, the inheritance gate, and the rule that Herald-side rules promote into the parent constitution via the HelixConstitution repo — not by editing the parent from inside Herald).

How Constitution Submodule rules and mandatory constraints are extended

We MUST EXTEND the `constitution` Submodule with the following rules and mandatory constraints which will make possible for us to do the extending:

- Any Submodule that has the `constitutable` directory or any directory containing it inside and that is located in the root of the Project and which contains the structure and files like the `constitution` Submodule has, it will be used for extensions and overrides of the top of the definitions provided by the `constitution` Submodule.
- Rules and mandatory constraints are loaded, evaluated and applied in the following priority: `constitution` Submodule → `constitutable` directories extensions and overrides for Constitution, `CLAUDE.md`, `AGENTS.md` and other definitions we support by the `constitution` Submodule → Project and Submodules Constitution, `CLAUDE.md` and `AGENTS.md` and other definition files defining rules and mandatory constraints.
- The `constitutable` directory can have multiple subdirectories with `constitution` Submodule layouts in it. For example all these paths are roots for extending or overriding `constitution` Submodule rules and mandatory constraints: `constitutable` (and all content directly in the root of the directory), `constitutable/flavor_1`,

`constitutable/flavor_2`, `constitutable/flavor_3/variant_1`, `constitutable/flavor_3/variant_2`. Each MUST HAVE in itself one of the mandatory files used for recognizing the `constitution` Submodule compatible definitions for rules and mandatory constraints: `Constitution.md`, `CLAUDE.md` or `AGENTS.md`.

Note: The `tests` we have now and which may exist in `constitution` Submodule MUST BE properly extended and updated once changes are applied and implementation(s) are improved and extended!

Note: Herald is **primarily** consumed as a Submodule of another Project; in that case access to the `constitution` Submodule is through the root of that project (cloned under `project_root/constitution`, or another subdirectory of the operator's choice). For **standalone development** of Herald, the `constitution` is cloned **alongside** Herald (sibling-clone) — current development setup uses this layout, with `constitution/` next to `Herald/` under the parent `Projects/` directory. See `../../guides/CONSTITUTION_INHERITANCE.md` §"Standalone development" and R-10.

Note: Carefully investigate the codebase of the `constitution` Submodule before any changes are applied! We MUST BE aware about every single detail - how it works, what are files there, with what purpose, what we have to update / extend and which new thing MUST BE added. Once everything is in-depth analyzed completely, then we can perform comprehensive changes to fulfill complete seamless integration with Herald project.

Workable items naming prefix

For all opened workable items for the Herald project (under Issues, Issues_Summary, Fixed, Fixed_Summary, Status and Status_Summary (for each existing content), etc.) use the following prefix: `HRD`. For example: `HRD-001`, `HRD-002`, etc.

Technology stack

Herald project and all flavors MUST BE written in Go. The whole implementation - the binary we distribute and use with all its dependencies MUST BE Containerized using the `containers` Submodule: https://github.com/vasic-digital/containers.

Main Database: Postgres, part of the main Container (Docker or Podman Compose stack). **In-Memory Database:** Redis, part of the main Container (Docker or Podman Compose stack).

All Container ports shall start with `24XXX` prefix (chosen to sit inside the IANA User Ports band — 1024–49151 — below the Linux ephemeral floor at 32768 and above all common service defaults; see R-01 for the rationale and the rejected `70XXX` proposal that exceeded the 65535 TCP/UDP maximum) so we eliminate conflicts possibility with other containers.

So basically, we will be using ports one by one: `24001`, `24002`, `24003`, etc.

All System (Herald) Containers names MUST start with prefix `herald`.

Commons

The following paragraphs define shared functionality and implementations among all Flavors of Herald.

The `commons` will contain the most generic abstractions and shared implementations which will be later inherited through the inheritance hierarchy.

Example:

```
commons -> commons level 1 -> commons level 2 -> ... -> commons level N ->
```

Common Messaging Herald

Common Messaging Herald (`commons_messaging`) is the `commons level 1` abstraction layer. Every Messaging Herald Flavor MUST offer support for several main integrations:

- Telegram
- Slack
- Max (max.ru)
- Email
- Markdown document with PDF and HTML export

For each of Messaging services user MUST provide required tokens, credentials or API keys depending on the platform. All details MUST BE documented in proper user guides and manuals with step by step instructions so users can easily obtain and provide required information.

All sensitive data like credentials, tokens or API keys MUST be in inside proper `.env` file (create for the documentation purposes `.env.example` file to illustrate everything that users can put there) which MUST BE Git ignored! System MUST BE capable to obtain all these environment variables from exported variables from `.bashrc` and `.zshrc` or any other System profile script which can export the variables. **Resolution order (12-factor aligned, per R-04):** (1) explicit CLI flag → (2) shell-exported env var → (3) value from `.env` (fallback only — does NOT override exported shell vars) → (4) compiled default. Operators who need the inverse ("file wins") can opt in via an explicit `--env-file-override` flag when the loader supports it.

Everything that is sent or received through any of the integrated Messengers channels such as Telegram, Slack, Max and others (Email as well) will be stored inside main Markdown file and exported into PDF and HTML regularly. Markdown file and its exports MUST BE always in sync (per Universal Constitution §11.4.61 + §11.4.65). Location of the Markdown file MUST BE `docs/herald/diary/main.md` (`main.pdf` and `main.html`) relative to the **operator-specified working directory** — defaulting to (a) Herald's own root when running standalone, or (b) the parent project's root when consumed as a submodule (discovered via the same parent-walk as `find_constitution.sh`). Override with `--diary-root <path>`. See R-20.

Messaging flow(s)

Messages we send (the data) to the channels (messengers integrations) MUST BE supported to work (if particular messenger channel allows this) all of the following scenarios:

- **Simple message** — we send content to the channel (textual), it is sent and displayed in messenger channel to all subscribers.

- **Message with attachment(s)** — we send the content with one or more attachments, it is sent and displayed in messenger channel to all subscribers which can then download the attachments.
- **Simple quote message** — we send content to the channel (textual) that is a reply to an existing channel message, it is sent and displayed in messenger channel to all subscribers (it can contain zero, one or more attachments which subscribers can download).

Subscribers

Subscribers are all users added to channels of the supported Messengers. They can communicate with the System! Users (Subscribers) can receive everything the System publishes and interact! For example, to a particular message containing some information a User (Subscriber) can reply, reply with an attachment, or they can send a brand new message with or without attachment.

Particular Flavors of Herald will have the understanding for the content received from Subscribers and about the ones we send towards them (with or without attachments).

APIs

We MUST perform in depth research and bring in all required APIs and SDKs required for each Messaging solution to be fully incorporated. We MUST perform deep web research and obtain information about API documentation and SDKs (Go). Every SDK and API which are available as Git repositories MUST BE incorporated as Git Submodules into the project. Example path for the Submodule(s): `commons_messaging/api/telegram` or `commons_messaging/sdk/telegram`.

We MUST fully integrate Max for Business (https://max.ru and https://business.max.ru/).

Same applies for Slack and Telegram.

Priority of integration: Telegram → Max → Slack.

For upcoming iterations we MUST document the following upcoming Messengers to be integrated: Microsoft Teams, Lark, Discord, WhatsApp, Viber. Additional channels may be added in later iterations.

For each platform (Messenger integration) we MUST perform in-depth web deep research and gather all documentation, articles, technical documentation and open-sourced codebases with official and unofficial APIs and SDKs and other components we could integrate.

We MUST make sure that all materials we gather and use MUST BE properly adapted and put in our own version of technical documentation under the `docs` directory into properly structured hierarchy (SDKs, APIs, and so on).

Flavors (the implementations)

Main Go abstractions and shared codebase (with shared implementations) will be used as the base which Flavors will inherit and build on top of it.

Project Herald

The Project Herald is focused on Projects and its development. All Projects share some commons and Project Herald MUST FIT as the universal player here!

What are the specifics that Project Herald is having and others do not?

Project Flavor of Herald is focused on Projects development and all development lifecycles. Main purpose is integrating it into the development cycles and pipelines (for example LLM driven).

System during the process gets into situations, certain events happen during the development lifecycle and these information are properly organized and formulated for sending to Subscribers.

Project Flavor recognizes most common scenarios during the regular development lifecycles and in what form to communicate the events and content towards the Subscribers (Users).

Project Flavor recognizes special commands and keywords that can be part of sent content (messages) towards the Subscribers and vice versa.

Project Flavor (like all other Flavors) MUST support proper mechanisms to detect validity of messages received from Subscribers.

Our System - Project Flavor listens for messages (and responses) from the Subscribers. Every received message from Subscribers side is processed. Some of received messages do require response, some may not.

Responding to messages MUST reference the parent message in the conversation (reply / quote) if any! If it exists (parent message) IT MUST BE REFERENCED!

In processing the whole thread of communication in replies / quotes (chained replies - all from bottom to the start of the particular thread) is fully taken into the account and parsed and processed!

Serious security validation is performed before any other steps are taken!

If Project does not define workable items tracking prefix, it will be determined by proper algorithm applied to project name and we will have the 3 letters prefix! We MUST create simple algorithm which will be doing the conversion from name to 3 letters prefix. For start we should do web research. Most likely open-sourced codebase exists somewhere and does this out of the box. We can incorporate or port it!

Inputs

All data we receive from Subscribers - messages content (fresh messages or part of threads), attachments users provide, are all Inputs.

Input commands

- `Bug:`, `Issue:` — Reporting problems
- `Query:`, `Request:`, `Question:` — Requesting the information or report
- **TBD**

Input attachments

TBD

Project Herald's Constitution rules and mandatory constraints

TBD

System Herald

TBD

What are the specifics that System Herald is having and others do not?

System Herald's Constitution rules and mandatory constraints

TBD

Others and misc

Here we will list main ideas for upcoming Flavors which MUST BE planned with proper deep web research and fully implemented:

- **TBD**

Specification documents

We MUST add into the Constitution (`Constitution.md`), `AGENTS.md` and `CLAUDE.md` of the Herald project itself the following rule / mandatory constraint related to this technical specification:

> **IMPORTANT:** Whenever this document (`docs/specs/mvp/specification.md`) or any under the `docs/specs` root directory and any of its children directories (any level deep) is modified, comprehensive planning and implementation of all changes is MANDATORY to be performed! This does not apply to new or renamed files! For new files we MUST explicitly tell the worker (CLI agent) what to do with the newly created / copied files!

Documentation

Make sure the main `README` document is fully updated with all relevant project details and all user guides and manuals are properly linked (and other documentation and relevant materials too) to it!

We MUST have all mandatory documentation up to the smallest details, full user guides and manuals, diagrams and scheme in all major formats and other relevant materials users may need.

Testing

Whole project and all of its derivatives MUST follow testing rules from our root Constitution (`Constitution.md`), `CLAUDE.md` and `AGENTS.md` (`constitution` Submodule from the main parent project).

Notes

- Many technical details which can be specified for particular Herald Flavor or certain specialization may be actually general-purposed! These all MUST be identified during the processing of this specification and planned as shared (between the Flavors - Commons, or as shared Components in the System)!

- We MUST pay attention which parts of the whole Project's codebase MUST be located in `commons` and how many (if any) `commons` layers are needed! This MUST BE carefully planned with a vision of potential growth - more Herald Flavors and extending functionalities.

Review (findings — to address)

> Added 2026-05-19 by automated review pass per user request. Each finding is a candidate for follow-up work; **nothing here is fixed yet** in the body of this spec. Items are grouped by category and tagged with a working ID (`R-NN` / `M-NN`) for later reference. Cross-document sync items were validated against `README.md`, `CLAUDE.md`, `AGENTS.md`, `docs/guides/HERALD_CONSTITUTION.md`, and `docs/guides/CONSTITUTION_INHERITANCE.md`.

Technical feasibility (must resolve before scaffolding)

- **R-01. Port range `70XXX` is invalid.** TCP/UDP ports max out at **65535**. Ports `70001+` cannot bind on any standard OS. §"Technology stack" needs to pick a feasible range — e.g., `60000–60999`, `62000–62999`, or `54000–54999` — and document conflict-avoidance vs. the Linux/Windows ephemeral range (`49152–65535`).
- **R-02. CLI-binary vs. listener-daemon contradiction.** §"How Herald should achieve…" frames flavors as CLI binaries for CI/cron, but §"Project Herald" says the system *listens* for messages from subscribers and processes inbound threads. These are different runtime models. Pick one of: (a) split into a CLI invoker + a long-running daemon (e.g., `pherald` CLI + `pherald-d` daemon); (b) CLI-only with each invocation polling for new inbound messages; (c) daemon-only with a CLI wrapper. Choice ripples into deployment, state storage, and the security-validation surface.
- **R-03. "Always in sync" Markdown ? PDF ? HTML is not literally achievable.** Real-time export on every inbound/outbound message is expensive (Pandoc/wkhtmltopdf). Define: sync trigger (per-message, batched every N seconds, on-commit?), failure semantics (does a failed PDF export fail the message send?), and the anti-bluff gate (how do we *prove* the three files match after every change?).
- **R-04. `.env` override order is inverted vs. 12-factor convention.** §"Common Messaging Herald" says shell-exported vars load **first** and `.env` **overrides** them. 12-factor convention is the opposite (shell-exported wins, `.env` is dev fallback). Confirm which is intended; the surprising override direction is a foot-gun for operators if it goes to production unflagged.
- **R-05. Delivery confirmation semantics undefined.** Anti-bluff requires every PASS to carry positive evidence. What counts as "delivered" per channel? Telegram `sendMessage` returns ok but cannot prove subscriber-read; SMTP `250 OK` proves only acceptance by the relay; Slack RTM returns send-ack only. Define per-channel evidence and the no-bluff gate **before** Telegram integration starts.
- **R-06. Max (max.ru) integration uncertainties.** Max is a Russian messenger (VK Group). Public bot/business API documentation availability, English-language coverage, and Go SDK availability all need a discovery pass before this can be planned. Consider also geopolitical/sanctions implications for a project also targeting GitHub/GitLab/etc.
- **R-07. Subscriber identity reconciliation across messengers.** A single human may be a subscriber on Telegram + Slack + Email simultaneously. There is no shared identity primitive across these platforms. Define: do we treat each `(channel, id)` as independent? Operator-side linking? Required for de-duplication and threading.
- **R-08. Reply/quote abstraction is non-uniform across channels.** Telegram, Slack, and Max each have first-class reply IDs but different APIs; Email replies are detected via `In-Reply-To` / `References` headers (lossy); Markdown export has no inherent thread.

Need a unified internal model + per-channel mapping before §"Project Herald" "chained replies" can be implemented.

Cross-document sync (mismatches with existing guides)

- **R-09. "Incorporate into the root Constitution" contradicts** `CONSTITUTION_INHERITANCE.md`**.** §"Integration into the Constitution" says we will eventually edit the root `Constitution.md`/`CLAUDE.md`/`AGENTS.md`. The inheritance guide §"Promoting Herald rules into the constitution" forbids editing the parent constitution from Herald; promotion goes through the HelixConstitution repo with a §11.4 + §11.4.10 universal-vs-project audit. Resolution: reword the spec to "promote via HelixConstitution PR" rather than "incorporate into root Constitution".
- **R-10. "ALWAYS incorporated as Submodule" excludes standalone development.** §"Integration into the Constitution" → 2nd Note states Herald is always a submodule of another project. `HERALD_CONSTITUTION.md` §104 and `CONSTITUTION_INHERITANCE.md` §"Standalone development" explicitly support sibling-clone for standalone work. Resolution: reword to "primarily consumed as a submodule; standalone work uses sibling-clone (see CONSTITUTION_INHERITANCE.md)".
- **R-11. SDK-as-Git-Submodule rule expands the currently-empty owned-submodule set.** §"APIs" mandates Git Submodules for all SDKs. `HERALD_CONSTITUTION.md` "Owned-submodule set" is currently `(none)`. Once we add `commons_messaging/sdk/telegram` etc., the constitution doc and the inheritance gate must be updated in lockstep. Also: Go ecosystem ergonomics favor `go.mod` for `go get`-able SDKs (e.g., `tucnak/telebot` for Telegram); reserve submodules for vendored/unofficial code that we patch.
- **R-12.** `containers` **submodule (§"Technology stack") same propagation requirement as R-11.** Adding `vasic-digital/containers` as a submodule will require updating the `Owned-submodule set` in `HERALD_CONSTITUTION.md` and re-running the inheritance gate.
- **R-13.** `constitutable/` **mechanism is described but not implemented.** §"How Constitution Submodule rules and mandatory constraints are extended" defines a discovery + override loader (apply order: `constitution` → `constitutable/**` → project docs). The directory exists (empty) and `CLAUDE.md` mentions it, but **no loader, no discovery script, no gate** enforces the contract. Plan: add a `find_constitutable.sh` companion, extend the inheritance gate to assert any `constitutable/<path>` contains at least one of `Constitution.md` / `CLAUDE.md` / `AGENTS.md`, and document the apply-order in `HERALD_CONSTITUTION.md`.
- **R-14. Spec-change rule not yet propagated to all Herald root docs.** §"Specification documents" mandates the rule be added to Herald's `Constitution.md` (i.e., `HERALD_CONSTITUTION.md`), `CLAUDE.md`, and `AGENTS.md`. As of this review: `CLAUDE.md` ✓ (added in current session); `AGENTS.md` × pending; `HERALD_CONSTITUTION.md` × pending. Also worth promoting to an I7 invariant in the inheritance gate (paired-mutation per §1.1).
- **R-15. Apply-order text has internal duplication.** §"How Constitution Submodule rules and mandatory constraints are extended" reads "`CLAUDE.md` and `CLAUDE.md`" — almost certainly intended to be "`CLAUDE.md` and `AGENTS.md`". Left as-is per the "do not modify content yet" instruction; flagged here for operator confirmation before fix.

Specification ambiguities (need operator decision before implementation)

- **R-16. "Serious security validation" is undefined** (§"Project Herald"). Open questions: threat model? Allowlist of subscriber IDs per channel? HMAC-signed messages? Reply-

from-known-thread-only? Rate limiting? Without this, the security validation step cannot be designed or tested.
- **R-17. 3-letter prefix collision handling.** §"Project Herald" describes a derived 3-letter prefix algorithm but says nothing about collisions across projects. Need: deterministic suffix? Operator override? Persistence so a project never silently re-derives a different prefix? Also: the prefix space is $26^3 = 17{,}576$ — collisions are inevitable at scale.
- **R-18. Multi-flavor binary versioning.** `pherald`, `sherald`, … share `commons`. How are flavors versioned? Lockstep (mono-repo `go.work`)? Independent SemVer with `commons` pinned? §"Technology stack" should pick one.
- **R-19. "Probably some more will be added"** (§"Common Messaging Herald — APIs") uses guessing language. Universal §11.4.6 forbids `probably`/`likely`/`maybe` when *reporting causes*. The spec context here is *planning*, not *cause-reporting*, so it's borderline — recommend deterministic phrasing anyway ("additional channels may be added in later iterations").
- **R-20. Diary file path is project-local but its scope is ambiguous.** §"Common Messaging Herald" pins `docs/herald/diary/main.md` inside "the Project". For a project that consumes Herald as a submodule, is this the **parent's** `docs/herald/diary/main.md`, or **Herald's own**? Affects how the path is resolved at runtime.
- **R-21. "He can just brand new message"** (§"Subscribers") is missing a verb — likely "send a brand new message". Left as-is per "do not modify content yet"; flagged for confirmation before fix.
- **R-22. Subscriber pronoun is gendered** (§"Subscribers", "he can just …"). Minor inclusivity nit; flag for operator decision on house style.

Markdown / structure improvements applied in this revision

- **M-01.** Added a **Table of Contents** at the top with explicit anchor slugs.
- **M-02. Promoted to top-level H2 sections** what were H3 under "Integration into the Constitution": `Workable items naming prefix`, `Technology stack`. They are project-wide conventions, not constitution-integration mechanics.
- **M-03. Promoted to top-level H2:** `Specification documents` (was nested under "Flavors → Specification documents"). The rule applies project-wide, not per-flavor.
- **M-04. Converted** `*Note:*` **lines into portable** `> **Note:**` **blockquote callouts** (rather than GitHub-specific `[!NOTE]`) so the document renders consistently across all four mirror hosts (GitHub, GitLab, GitFlic, GitVerse).
- **M-05.** Used `> **IMPORTANT:**` for the spec-change rule for the same portability reason.
- **M-06. Replaced bare** `Tbd` **markers with** `**TBD**` **(bold)** for visual distinction from prose.
- **M-07. Switched non-rendering Markdown links of** `git@…` **URLs** (which don't form valid HTTP URLs and don't render as proper links in any of the four mirror hosts' Markdown renderers) into **code spans** for SSH URLs and **autolinks** (`<https://…>`) for HTTPS URLs.
- **M-08.** Added an **Upstreams** H2 heading where there was previously only an unlabeled list — lets the ToC link to it.
- **M-09.** Added **cross-links** from §"Integration into the Constitution" to `HERALD_CONSTITUTION.md` and `CONSTITUTION_INHERITANCE.md` so readers find the authoritative inheritance contract immediately.
- **M-10.** Inserted a horizontal rule (`---`) before the Review section to visually separate spec body from review findings.
- **M-11.** Bolded **Priority of integration** and the per-flow names ("Simple message", "Message with attachment(s)", "Simple quote message") for scanability.

Typos fixed (silent corrections, no semantic change)

The following spelling/grammar errors were corrected in place. Reverse-able via `git diff`; flagged here for transparency.

#	Original	Corrected
T-01	`exiting project upstreams`	`existing project upstreams`
T-02	upstream label `GitFlic` (2nd, for `gitverse.ru`)	`GitVerse`
T-03	`Heralds is then sending`	`Herald is then sending`
T-04	`binnary`, `binarries`	`binary`, `binaries`
T-05	`throguh`	`through`
T-06	`Crond - cron jobs`	`cron jobs`
T-07	`Submoudule` (×5+)	`Submodule`
T-08	`defintions` (×2)	`definitions`
T-09	`existi`	`exist`
T-10	`parnet`	`parent`
T-11	`fullfill`	`fulfill`
T-12	`Herlad`	`Herald`
T-13	`contet`	`content`
T-14	`writtn`	`written`
T-15	`woth`	`with`
T-16	`hierachy`	`hierarchy`
T-17	`Messeging` (×2)	`Messaging`
T-18	`fron`	`from`
T-19	`variabels`	`variables`
T-20	`interated Messengers`	`integrated Messengers`
T-21	`we sent the content`	`we send the content`
T-22	`attchments` (×2), `attchemtns`	`attachments`
T-23	`containing`	`containing`
T-24	`the once we send`	`the ones we send`
T-25	`bring int`	`bring in`
T-26	`unoffcial`	`unofficial`
T-27	`Flawor`	`Flavor`
T-28	`MUS BE`	`MUST BE`
T-29	`qutes`, `chined`	`quotes`, `chained`
T-30	`defint`	`define`
T-31	`algorhythm` (×2)	`algorithm`
T-32	`opensourced` (×2)	`open-sourced`
T-33	`Tbd` (×6)	`**TBD**`
T-34	`derrivates`	`derivatives`
T-35	`tehcnical`	`technical`
T-36	`grofth`	`growth`
T-37	`functinalites`	`functionalities`

Recommendations (research-backed proposals)

> Added 2026-05-19. Each `R-NN` below proposes a concrete approach with references, derived from web research (cited per item) and from desk review of the existing project guides (`HERALD_CONSTITUTION.md`, `CONSTITUTION_INHERITANCE.md`, `CLAUDE.md`, `AGENTS.md`, `README.md`). Where a number is approximate (e.g. GitHub stars), it is order-of-magnitude only; re-verify at the moment of pinning.
>
> **Applied 2026-05-19 (text-level fixes, this commit):** R-01 (port range 24XXX), R-04 (.env precedence reverted to 12-factor), R-09 (constitution promotion wording), R-10 (sibling-clone for standalone), R-15 (`CLAUDE.md` and `AGENTS.md` duplicate fix), R-19 (deterministic phrasing for "Probably some more"), R-20 (diary path scope clarification), R-21 (missing verb), R-22 (gendered pronoun). R-14 partially applied: spec-change rule propagated to Herald `AGENTS.md` + `HERALD_CONSTITUTION.md` §106, gate invariant I7 added.
>
> **Deferred to focused next rounds:** R-02 (CLI/daemon split), R-03 (Markdown sync strategy), R-05 (delivery enum), R-06 (Max integration), R-07 (subscriber identity), R-08 (reply abstraction), R-11/R-12 (submodule additions when scaffolding starts), R-13 (`constitutable/` loader + I8/I9 gates), R-16 (security pipeline), R-17 (prefix algorithm in `commons/prefix`), R-18 (multi-binary versioning).

R-01 — Container host port range

Recommendation: Use `24000-24999` as Herald's container host-port allocation range. Reserve sub-blocks per service family: `24000-24099` for app binaries, `24100-24199` for Postgres, `24200-24299` for Redis, etc. Avoid the `7XXXX` range (invalid — TCP/UDP max is 65535), avoid `49152-65535` (IANA Dynamic / Private, doubly unsafe because Docker 20+ picks ephemerals from `49153-65535` ignoring the kernel sysctl), and stay below the Linux ephemeral floor at `32768`.

Why: Linux's default `ip_local_port_range` is `32768-60999`, so any *fixed* host port in that window may collide with an outgoing ephemeral socket. IANA RFC 6335 reserves `49152-65535` as Dynamic/Private. `24000-24999` sits inside the User Ports band (1024-49151), above all common service defaults (Postgres 5432, Redis 6379, web 3000/5000/8000/8080/9000/9090/9200, memcached 11211, Mongo 27017), and below 32768 — clear of both ephemeral and well-known service defaults.

References:

- Linux kernel IP sysctl docs — confirms `ip_local_port_range` default `32768 60999`.
- RFC 6335 — IANA port registry procedures — defines System (0-1023), User (1024-49151), Dynamic (49152-65535) bands.
- moby/moby#43054 — Docker 20+ ignores `ip_local_port_range` and picks ephemerals from 49153-65535.

R-02 — CLI vs. daemon split

Recommendation: Ship Herald as a **single binary per flavor with Cobra subcommands**: `herald send …` (one-shot, exits on completion) for CI/cron, `herald serve` (long-running listener) for inbound replies, `herald <flavor-specific> …` for flavor commands. Both

modes share config loader, channel adapters, and storage layer; only the entry point and lifecycle differ.

Why: Every Go project surveyed that supports both invoke-and-exit and listen-long-running uses one binary with subcommands — eliminates duplicate config/auth/logging code and gives operators one artifact per flavor. Vault: `vault server` vs `vault kv put`. Gitea: `gitea web` vs `gitea admin`. Caddy: `caddy run` vs `caddy reload`. Two binaries would force duplicate plumbing and double the container-image count. For CI: `herald send` connects, transmits, awaits ack with a deadline, exits non-zero on failure (no polling loop). For inbound replies: `herald serve` blocks on per-channel subscriber loops.

References:

- hashicorp/vault `command/agent.go` — subcommand registered via the same root CLI as `vault kv`, `vault login`.
- caddyserver/caddy `cmd/commands.go` — factory-pattern Cobra registration, all subcommands compiled into one binary.
- go-gitea/gitea `cmd/` package — explicit `web` and `admin` subcommands on a single static binary.
- spf13/cobra — the de-facto Go CLI framework these projects all use.

R-03 — Markdown ? PDF ? HTML sync strategy

Recommendation: Use **Pandoc** as the converter (Markdown → HTML; HTML → PDF via **WeasyPrint** as Pandoc's `--pdf-engine`). Trigger rebuilds via **fsnotify-watched debounced builds** (500 ms idle window, 2 s ceiling) under `herald serve`, and via a **post-write hook** in the diary writer under one-shot mode. Maintain `docs/herald/diary/.sync.json` = `{md_sha256, html_sha256, pdf_sha256, source_md_sha256_at_build, built_at}`. The anti-bluff gate: (a) read current `main.md` SHA-256; (b) assert it equals `source_md_sha256_at_build` in the manifest; (c) for HTML, re-run Pandoc to a temp file and assert byte-equality with the on-disk HTML; (d) for PDF, compare a normalised text extraction (`pdftotext main.pdf -`) against a deterministic extraction of the HTML body — byte-equal PDF comparison fails because PDF embeds creation timestamps and non-deterministic font subsetting.

Why: Pandoc is the only widely-used converter that reads CommonMark and emits both HTML and PDF with consistent styling. WeasyPrint is preferred over wkhtmltopdf (deprecated, ignores CSS page sizes). mdbook is book-shaped, Marp is slides-shaped — neither fits an append-only diary. fsnotify alone misfires (a single editor save fires 4-12 events); debouncing with a ceiling is the documented best practice. The manifest + re-render-and-compare gate makes "I forgot to rebuild" *provably* a CI failure rather than silent drift.

References:

- Pandoc User's Guide — PDF engines — lists `weasyprint`, `wkhtmltopdf`, `tectonic` as `--pdf-engine` options.
- fsnotify/fsnotify — canonical Go file watcher; docs warn editors emit bursts requiring debounce.
- zoni/obsidian-export — precedent for "render Markdown vault to a canonical form on every change."

R-04 — `.env` precedence

Recommendation: Align Herald with 12-factor (shell wins, `.env` is the fallback) and revise the spec. Use `github.com/joho/godotenv`'s `Load()` (default, non-overriding) rather than `Overload()`. Document the precedence chain in the spec: (1) explicit CLI flag → (2) shell-exported env var → (3) `.env` file → (4) compiled default. If operators ever need the inverse ("file wins"), expose it through an opt-in flag `--env-file-override` mapping to `godotenv.Overload()`.

Why: Every mainstream Go env loader either defaults to shell-wins or is configurable to it. `godotenv.Load()` skips already-set vars; `Overload()` is opt-in and labeled "use with caution" in the README. `kelseyhightower/envconfig` reads only the live environment (no `.env` concept). `caarlos0/env` reads only `os.Getenv`. `spf13/viper`'s `AutomaticEnv` puts env vars above file configs. `knadh/koanf` makes precedence whichever `Load()` you call last. The spec's current order breaks operators' escape hatch (`HERALD_TELEGRAM_TOKEN=xxx herald send …` should win over a stale `.env`) and breaks Docker/Kubernetes secret injection.

References:

- joho/godotenv README — "Existing envs take precedence of envs that are loaded later."
- kelseyhightower/envconfig — env-only by design.
- caarlos0/env — parses live env into structs.
- spf13/viper — documents canonical precedence order with env vars above config files.
- Twelve-Factor App — Config — the convention Herald should match.

R-05 — Per-channel delivery-confirmation semantics

Recommendation: Adopt a four-level evidence enum: `Accepted | Routed | Delivered | Read`. Per-channel ceilings: **Telegram** → `Routed` (Message struct = stored & queued; `Read` only via Business Bot connection with `can_read_messages` right, or MTProto); **Slack** → `Routed` (`ok: true` + `ts` proves posted to channel, no read receipts); **Email/SMTP** → `Accepted` by default, upgradable to `Delivered` by parsing RFC 3464 DSNs from the bounce mailbox, `Read` only when a recipient voluntarily returns an RFC 8098 MDN; **Max** → `Routed` (parity with Telegram per the dev.max.ru docs). Persist the highest evidence level observed per send; emit a PASS only when the configured per-tenant floor is reached.

Why: Anti-bluff requires positive evidence. SMTP 250 is a hop-by-hop ACK, not subscriber delivery — DSN is the only standards-based proof. Telegram and Slack Bot/Web APIs do not expose user-side read events to apps; treating "API returned OK" as `Delivered` would be a lie.

References:

- Telegram Bot API — sendMessage — returns Message on success; no read receipts in standard Bot API.
- Telegram Bot API — connected business bots — read marking requires Business connection + `can_read_messages`.
- Slack chat.postMessage — `ok: true` + `ts` proves Slack stored & broadcast; no per-user read event.
- RFC 3464 — DSN — machine-parseable delivery status from MTAs.
- RFC 8098 — MDN — recipient-side Read disposition, voluntary.

R-06 — Max.ru integration viability

Recommendation: Implement Max as a first-class channel using the **official Bot API** at `dev.max.ru` via the Go client `github.com/max-messenger/max-bot-api-client-go` (Apache-2.0, actively maintained as of May 2026). Bot registration is via in-app `MasterBot`. **Gate Max behind a build tag** (`herald_max`) and a documented sanctions advisory in the operator manual: VK Group's parent (`VK Company Limited`) is not on the OFAC SDN list at time of writing, but several VK-affiliated individuals are designated, and EU restrictive measures evolve frequently. Operators in sanctioned jurisdictions or US persons should consult counsel before enabling.

Why: A public, Apache-2.0 Go SDK with English+Russian docs and a typed API exists, so clean integration is feasible (no reverse engineering). The geopolitical risk is an operator-deployment concern, not a code-correctness concern — ship it disabled-by-default with a clear advisory rather than refuse to support a documented public API. Delivery-evidence ceiling appears equivalent to Telegram (API ack = `Routed`).

References:

- max-messenger/max-bot-api-client-go — official Go SDK, Apache-2.0, English README.
- dev.max.ru — official developer portal.
- OFAC Recent Actions — primary source operators must check at deploy time.
- Linux Foundation: Open Source and OFAC — publishing OSS is generally outside OFAC's scope; deployment may not be.

R-07 — Subscriber identity reconciliation across messengers

Recommendation: Adopt the **matterbridge model as the default** — per-channel-id is the source of truth, with a configurable display-name template (`RemoteNickFormat`-style) for cross-posts. Layer **operator-mapped linking** on top via a `SubscriberAlias{ subscriber_id, channel, channel_user_id, verified_at }` table. Reject Matrix-style "ghost puppeting" — that model assumes a unifying meta-protocol Herald is not. Provide a self-claim flow (subscriber DMs a one-time token to the bot on each channel) only for tenants that opt in.

Why: matterbridge deliberately does no cross-channel identity unification — each user is `{NICK}@{PROTOCOL}` and only display formatting bridges them; this is the right minimum for a fan-out tool. Matrix `mautrix-*` bridges use double-puppeting because they need bidirectional bridging into a unifying user namespace; Herald fans out, so the complexity isn't justified. ntfy and Gotify don't reconcile at all, confirming "no reconciliation" as a viable baseline.

References:

- matterbridge Gateway config — `RemoteNickFormat`, `{NICK}@{PROTOCOL}`; no cross-channel user table.
- mautrix double-puppeting — tightly coupled to Matrix.
- ntfy config — ACLs — topic-scoped, no cross-channel identity.
- gotify/server user model — server-local users only.

R-08 — Unified reply/quote thread abstraction

Recommendation: Define an internal value type `ConversationRef { Channel ChannelID; ThreadID string; ParentMessageID string; RootMessageID`

`string }`, plus a per-channel `ThreadAdapter` interface implementing `Open(ref) → channel-native handle` and `ReplyHeaders(ref) → channel-native fields`. Mapping rules: Slack `thread_ts` → `ThreadID = RootMessageID`; Telegram `reply_to_message_id` → `ParentMessageID` (`ThreadID == ""` outside forum topics); Email `In-Reply-To` → `ParentMessageID`, `References` → ordered ancestor chain with `RootMessageID = References[0]`; Markdown export emits a blockquote prefix with the parent's `RootMessageID` as a stable anchor. Persist `(tenant, logical_thread_id) → ConversationRef` in a `thread_refs` table — Slack `thread_ts` is per-channel, Telegram message IDs per-chat, Email `Message-ID` global; there is no derivation that works without a lookup table.

Why: matterbridge issue #638 is still open precisely because there is no clean cross-protocol thread primitive — but the protocols' models are well understood individually. Splitting `ThreadID` from `ParentMessageID` captures Slack's flat-thread model and Email's tree model without forcing either to lie.

References:

- matterbridge #638 — Thread/reply preservation — confirms no clean universal mapping exists.
- Slack — Retrieving messages (threads) — `thread_ts` equals parent `ts`.
- Telegram Bot API — sendMessage (reply_parameters) — `reply_to_message_id` per-chat; `message_thread_id` only for forum topics.
- RFC 5322 §3.6.4 — `Message-ID`, `In-Reply-To`, `References` semantics.

R-05–R-08 — Go SDK survey

Channel	Recommended SDK	Stars (approx)	Last release	License	Maintenance	Ergonomics
Telegram Bot	`gopkg.in/telebot.v3` (tucnak/telebot)	~4.5k	v3.x active (Bot API ≥ 7.1)	MIT	Active	Highest-level routing/ middleware API; opinionated, less 1:1 with Bot API.
Telegram Bot (alt)	`github.com/mymmrac/telego`	~0.8k	Apr 2026	MIT	Active	1:1 Bot API types, fasthttp by default; pick this for raw fidelity.
Telegram Bot (alt)	`github.com/go-telegram/bot`	(imported-by ~384)	Mar 2026	MIT	Active	Zero-dependency, Bot API 9.5; easy to vendor.
Slack	`github.com/slack-go/slack`	several k	rolling	BSD-2-Clause	Active	Covers REST + RTM/Socket Mode; community default.
Discord	`github.com/bwmarrin/discordgo`	~5.7k	v0.29.0 May 2025	BSD-3-Clause	Sustainable	Low-level bindings, near-complete API + voice + gateway.

Channel	Recommended SDK	Stars (approx)	Last release	License	Maintenance	Ergonomics
Microsoft Teams	`github.com/atc0005/go-teams-notify/v2`	smaller	Active	MIT	Active	Incoming-webhook-only; **no official Microsoft Go SDK** — use webhooks/ Workflows for fan-out.
Lark / Feishu	`github.com/larksuite/oapi-sdk-go`	~0.5k	Active (v3)	MIT	**Official**, active	Code-gen flavor; verbose but accurate.
WhatsApp (Web multi-device)	`go.mau.fi/whatsmeow` (`tulir/whatsmeow`)	~5.5k	Rolling	MPL-2.0	Active	Reverse-engineered WA Web; **not** for WA Business API.
WhatsApp (Business Cloud API)	`github.com/twilio/twilio-go`	~0.7k	Active	MIT	Official	Multi-product Twilio SDK; WA via Twilio Senders.
Viber	**no official Go SDK**	—	—	—	—	Community: `mileusna/viber`, `strongo/bots-api-viber`. Recommend hand-rolled REST client.
Email SMTP (high-level)	`github.com/jordan-wright/email`	~3k	Stable	MIT	Low activity, stable	Highest-level "email for humans"; attachments + pooling.
Email SMTP (low-level)	`github.com/emersion/go-smtp`	~2k	Active	MIT	Active	Pair with `go-message` for MIME and `go-msgauth` for DKIM.
Email (vendor)	`github.com/sendgrid/sendgrid-go`	~1k	Active	MIT	Official	Pairs with SendGrid Event Webhook for Delivered/ Bounce/Open evidence (R-05).
Max	`github.com/max-messenger/max-bot-api-client-go`	~0.08k	v1.6.17 May 2026	Apache-2.0	Active	Typed Bot API client; English README.

Star counts are order-of-magnitude estimates from public registries; re-verify before pinning in `go.mod`.

R-09 — "Incorporate into root Constitution" wording

Recommendation: Reword §"Integration into the Constitution" to: *"Rules that mature into universal status are promoted via a HelixConstitution PR after the §11.4 + §11.4.10 universal-vs-project audit, never by editing the parent constitution from Herald."* Then add a sentence pointing to `CONSTITUTION_INHERITANCE.md` §"Promoting Herald rules into the constitution" as the authoritative process.

Why: Aligns the spec with the documented inheritance contract. The current text could be read as authorising direct edits to the parent `constitution/` from a Herald PR — which the inheritance guide explicitly forbids.

References:

- `docs/guides/CONSTITUTION_INHERITANCE.md` §"Promoting Herald rules" — authoritative process.
- `docs/guides/HERALD_CONSTITUTION.md` §104 — no embedded constitution.

R-10 — "ALWAYS submodule" wording

Recommendation: Reword the Note: *"Herald is primarily consumed as a Submodule of another Project; in that case access to the* `constitution` *Submodule is through the root of that project. For standalone development, the* `constitution` *is cloned alongside Herald (sibling-clone); see* `CONSTITUTION_INHERITANCE.md` *§Standalone development."*

Why: Acknowledges sibling-clone path that the inheritance gate already supports (and that the current development setup actually uses, per the same Note).

R-11 — SDK-as-submodule expansion

Recommendation: Before adding the first SDK submodule (`commons_messaging/sdk/telegram` etc.), update `HERALD_CONSTITUTION.md` "Owned-submodule set" in the same PR, run the inheritance gate, and consider extending the gate with an invariant `I7` that asserts each entry in the owned-submodule set actually exists as a configured submodule. Reserve **submodules** for vendored/unofficial code that we patch; prefer `go.mod` for `go get`-able SDKs (e.g., `tucnak/telebot`, `slack-go/slack`, `bwmarrin/discordgo`) — submodule-everything bloats clone time and complicates dependency upgrades.

Why: Inheritance contract requires the "Owned-submodule set" to stay accurate. Go ecosystem ergonomics favor modules over submodules; mixing both is fine as long as the rule is documented.

R-12 — `containers` submodule adoption

Recommendation: Same as R-11: add `vasic-digital/containers` to `HERALD_CONSTITUTION.md` "Owned-submodule set" in the PR that introduces it. Defer adoption until containerization work begins — Herald is pre-implementation and there's no Compose file to wire yet.

References: `vasic-digital/containers`.

R-13 — `constitutable/` loader + gate

Recommendation: Implement the `constitutable/` discovery and apply-order in a small companion script `tests/test_constitutable.sh` (mirror the structure of `test_constitution_inheritance.sh`). Add gate invariant **I7**: *for every directory under* `constitutable/` *that contains at least one of* `Constitution.md`/`CLAUDE.md`/`AGENTS.md`*, all sibling files MUST belong to that recognized set.* Add gate invariant **I8** (paired with a §1.1 mutation test): the apply-order documented in `HERALD_CONSTITUTION.md` matches what the loader actually does. Implementation order: write the spec for the loader in `HERALD_CONSTITUTION.md` first (project-article §106), then I7 in the gate, then the loader, then I8 + mutation meta-test.

Why: Spec §"How Constitution Submodule rules and mandatory constraints are extended" describes an apply-order but no enforcement exists. The constitution's anti-bluff rule §1.1 requires every gate to have a paired mutation; the loader is no exception.

R-14 — Spec-change rule propagation

Recommendation: In the next docs PR, add the spec-change rule to `AGENTS.md` and `HERALD_CONSTITUTION.md`. Add gate invariant **I9** (paired): *Herald's* `AGENTS.md`*,* `CLAUDE.md`*, and* `HERALD_CONSTITUTION.md` *each contain the spec-change rule anchor* (e.g. the string `Whenever this document (\docs/specs/mvp/specification.md`)`). Mutation: remove the line from one file, expect FAIL.

Why: The spec mandates the rule live in all three; CLAUDE.md has it, the other two don't yet. Without a gate, drift will silently re-open the gap.

R-15 — "`CLAUDE.md` and `CLAUDE.md`" duplication

Recommendation: Confirm with operator and apply: replace "`CLAUDE.md` and `CLAUDE.md`" with "`CLAUDE.md` and `AGENTS.md`" in §"How Constitution Submodule rules and mandatory constraints are extended". This is a content correction (the user instructed "do not modify content yet"), so flag-and-wait rather than silent fix.

R-16 — Inbound-message security validation

Recommendation: Three-layer pipeline executed *before* any routing logic. **(1) Transport:** per-channel signature verifier — for Slack, `slack.NewSecretsVerifier` (HMAC-SHA256 over `v0:{timestamp}:{body}`, compared with `X-Slack-Signature` via `crypto/hmac.Equal`, rejecting timestamps older than 5 min); for Telegram, compare `X-Telegram-Bot-Api-Secret-Token` against the per-bot secret set via `setWebhook`; for generic webhook sources, HMAC-SHA256 against `X-Hub-Signature-256`; for Email, `emersion/go-msgauth/dkim` + SPF + DMARC alignment. All verifiers MUST use constant-time comparison and a 300 s monotonic-clock skew window. **(2) Sender:** per-channel allowlist keyed by canonical user-id (Slack `team_id+user_id`, Telegram `chat_id`, email DKIM-aligned `From`) loaded from `.herald/subscribers.toml`; unknown senders enter a quarantine queue, never the live fan-out. **(3) Content:** parse commands (`Bug:`, `Query:` …) with a strict regex-anchored tokenizer; never shell out, never template user input into shell/SQL; treat command bodies as length-bounded UTF-8 (e.g., 8 KiB).

Why: All three messengers publish official server-side signing primitives. Transport verification alone leaves the door open to a compromised-but-authenticated bot relaying attacker traffic, hence layer 2. Layer 3 is the standard defense against the spec's "no shell injection" requirement.

References:

- Slack — Verifying requests from Slack — `v0:timestamp:body` HMAC-SHA256, 5-min replay window.
- GitHub — Validating webhook deliveries — `X-Hub-Signature-256`, constant-time compare.
- Telegram Bot API — setWebhook — `secret_token` 1–256 chars, `X-Telegram-Bot-Api-Secret-Token` header.
- slack-go/slack `security.go` — reference Go implementation; pin ≥ v0.23.1 (GHSA-gxhx-2686-5h9g fixed empty-secret bypass).
- emersion/go-msgauth — DKIM (RFC 6376), DMARC, Authentication-Results for inbound email.

R-17 — Project name → 3-letter prefix algorithm

Recommendation: Ship a small deterministic `Prefix(name string) string` in `commons/prefix` (~80 LOC):

1. Normalize: Unicode-NFKD, strip diacritics, retain `[A-Za-z0-9]`, split on CamelCase boundaries and `[-_ /]` into tokens.
2. **Rule A (≥3 tokens):** first letter of each of the first three tokens — `HeraldRouterCore` → `HRC`.
3. **Rule B (2 tokens):** first letter of token 1, first letter of token 2, first internal consonant of token 2 — `HeraldRouter` → `HRT`, `HeraldRunner` → `HRN`.
4. **Rule C (1 token):** first letter, first internal consonant, last consonant — `Herald` → `HRD`.
5. Uppercase.
6. **Collision resolution:** maintain committed `.herald/prefix.lock` (TOML, `name → prefix`). On collision with a *different* project, compute `fnv1a32(name) mod 26` and replace the third letter with `'A' + (h mod 26)`; iterate up to 26 times, then fall back to numeric suffix `HR0`…`HR9`.

Why: No mature Go library generates *3-letter* abbreviations from arbitrary project names — the only Go prior art (`Defacto2/releaser/initialism`) is a curated lookup table, not a generator. The 17,576-slot space is large relative to one operator's universe, so collisions are operational, not statistical; deterministic + persisted-lock is sufficient. CamelCase + consonant heuristic handles the `Router`/`Runner` case without user input.

References:

- Defacto2/releaser/initialism — Go prior art; lookup-only, *negative* reference.
- FNV-1a hash (Wikipedia) — good dispersion on short, near-identical strings.
- Go `hash/fnv` — stdlib implementation.

R-18 — Multi-binary Go versioning

Recommendation: Single-repo, multi-module, **lockstep-versioned** layout:

```
Herald/
  go.work                 # local dev only, .gitignore'd
  commons/    go.mod      # module github.com/.../herald/commons
  pherald/    go.mod      # module github.com/.../herald/pherald  → cmd/phe
  sherald/    go.mod      # module github.com/.../herald/sherald  → cmd/she
  .goreleaser.yaml        # one config, builds all flavors
```

Each flavor's `go.mod` declares `require github.com/.../herald/commons vX.Y.Z`. One git tag `vX.Y.Z` triggers GoReleaser to build all flavors and tag `commons/vX.Y.Z` (Go modules require tag-name == module-path-suffix). Use `release-please` in `manifest` mode to bump all modules from Conventional Commits. `go.work` (`go work init ./commons ./pherald ./sherald`) keeps local dev fast; `.gitignore` **it** so CI builds each module against its declared `commons` version — that's what catches forgotten bumps.

Why: Kubernetes' `staging/` model is overkill at Herald's size and depends on a custom `publishing-bot`. GoReleaser's per-subproject `tag_prefix` (Pro feature) is the alternative for *independent* SemVer but complicates upgrade paths when `commons` changes. Lockstep + `go.work` is the pattern HashiCorp, Hugo, and most mid-size Go monorepos converge on.

References:

- Go modules — Workspaces — `go.work` semantics.
- Go tutorial: multi-module workspaces — official walkthrough.
- GoReleaser — Monorepo — `tag_prefix` for per-subproject SemVer (Pro feature).
- GoReleaser — Builds — auto-discovers `func main()` packages.
- googleapis/release-please — manifest-mode Conventional-Commits automation.
- Kubernetes `staging/` — heavyweight alternative reference.

R-19 — Guessing-language removal

Recommendation: Replace "Probably some more will be added." with "Additional channels may be added in later iterations." Apply globally — grep the spec for `probably`, `likely`, `maybe`, `seems`, `appears` and replace with deterministic phrasing.

Why: Universal §11.4.6 forbids guessing language when reporting causes; spec-planning prose is borderline but easy to fix preemptively.

R-20 — Diary path scope

Recommendation: Resolve `docs/herald/diary/main.md` relative to the **operator-specified working directory**, defaulting to (a) Herald's own root when running standalone, (b) the parent project's root when consumed as a submodule (discovered via the same parent-walk as `find_constitution.sh`). Expose `--diary-root <path>` flag to override. Document in the spec.

Why: The current "inside the Project" wording is ambiguous when Herald is a submodule. The parent-walk discovery pattern is already proven by the inheritance gate, so reuse it.

R-21 — "He can just brand new message" missing verb

Recommendation: Confirm with operator and apply: replace "he can just brand new message" with "he can send a brand new message". Tiny content correction held back per the "do not modify content yet" instruction.

R-22 — Gendered pronoun

Recommendation: Replace "he can send a brand new message" (after R-21) with "they can send a brand new message". Apply the same convention throughout the spec. Singular *they* is widely

accepted in modern technical writing and is consistent with the rest of the spec's gender-neutral subject usage ("User", "Subscriber").

Why: Inclusivity nit; trivially applied; no semantic change.